# Kubernetes: Environment Variables and Configuration Management

VISHWANATH M S
VISHWACLOUDLAB.COM

# Agenda

1. Plain Key-Value Environment Variables
2. ConfigMap as Environment Variables
3. ConfigMap as Volumes
4. Secret as Environment Variables
5. Secret as Volumes

# Plain Key-Value Environment Variables

These are static values directly defined in the Pod specification.

Use case: For hardcoded or small configuration values such as ENV=prod.

# Overview of Environment Variable

Environment variables are key-value pairs used to configure containers.

Injected at runtime without changing application code.

Used for configuration, secrets, toggles, and ports.

Sources: Plain key, ConfigMap, Secret.

# Lab 1: Plain Key as Variable in Pod

These are static values directly defined in the Pod specification.

Verify the variable inside the container.

Use case: For hardcoded or small configuration values such as ENV=prod.

# Example: Plain ENV Variable

```
env:
- name: ENV_MODE
  value: "production"
```

# YAML: Pod with Plain Environment Variable

```
apiVersion: v1
kind: Pod
metadata:
  name: env-var-pod
spec:
  containers:
  - name: mycontainer
    image: nginx
    env:
    - name: ENV_MODE
      value: "production"
```

# Lab 2: ConfigMap as Variable in Pod

Create a ConfigMap using literals.

Inject the ConfigMap into the Pod as environment variables.

ConfigMaps allow you to decouple configuration artifacts from image content.

# Example: ConfigMap Env

```
envFrom:
- configMapRef:
    name: my-config
```

# Command + YAML: ConfigMap Env

```
kubectl create configmap app-config --from-literal=APP_MODE=debug --
from-literal=APP_PORT=8080
```

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-env-pod
spec:
  containers:
  - name: app-container
    image: busybox
    command: ["sh", "-c", "env; sleep 3600"]
    envFrom:
    - configMapRef:
        name: app-config
```

# Lab 3: ConfigMap as Volume

Mount ConfigMap as file in container.

Useful for app configuration files.

Each key in the ConfigMap becomes a file with the corresponding value as content.

# Example: ConfigMap Volume

```
volumes:
- name: config-volume
  configMap:
    name: my-config-vol
volumeMounts:
- name: config-volume
  mountPath: /etc/config
```

# Command + YAML: ConfigMap Volume

```
kubectl create configmap
my-config-vol --from-
literal=app.conf="log.lev
el=info"
```

```
apiVersion: v1
kind: Pod
metadata:
  name: config-vol-pod
spec:
  containers:
  - name: busybox
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
  volumes:
  - name: config-volume
    configMap:
      name: my-config-vol
```

# ConfigMap as Volume

- ✅ **Purpose**: Mount configuration data as files directly into a container's file system, ideal for apps expecting files (not env vars).

- 📁 **File Structure**:
- Each **key** in the ConfigMap is represented as a **file name**.
- The **value** of the key becomes the **content of the file**.

- 🧩 **Use Cases**:
- Inject configuration files like app.conf, logging.conf, etc.
- Decouple application configuration from container image.

- 🛠️ **Benefits**:
- Dynamically update mounted files by updating the ConfigMap (with restart).
- Supports fine-grained access and structure (e.g., mount specific keys).

# ConfigMap as Volume -- Best Practices:

- Use subPath to mount specific **keys/files** from the ConfigMap.
- Mark the volume readOnly: true for better security.
- Use in tandem with readiness/liveness probes for safe reloads.

```
volumes:
  - name: config-volume
    configMap:
      name: my-config-vol
volumeMounts:
  - name: config-volume
    mountPath: /etc/config
    readOnly: true
```

# Lab 4: Secret as Variable in Pod

Secrets are intended to hold sensitive data such as passwords, tokens, or keys.

They can be exposed to containers as environment variables for secure access.

# Lab 4: Secret as Variable in Pod

**Purpose**: Manage **sensitive data** such as passwords, tokens, SSH keys, and certificates securely in Kubernetes.

**Storage**:

Stored in **base64 encoded** format (not encrypted by default).

Can be configured to be encrypted at rest using **KMS (Key Management Services)**.

**Usage in Pods**:

Inject secrets as **environment variables** to avoid hardcoding sensitive data in manifests or images.

Accessible within the container's runtime using standard environment access ($VAR_NAME).

# Lab 4: Secret as Variable in Pod

**Benefits**:

a. Prevents accidental exposure of secrets in version control.
b. Scoped access: secrets can be tightly controlled using RBAC and namespace boundaries.
c. Safer than ConfigMaps for storing confidential data.

**Best Practices**:

a. Use readOnly and minimal RBAC access.
b. Rotate secrets regularly.
c. Monitor and audit secret usage through tools like OPA or Gatekeeper.

# Lab 4: Secret as Variable in Pod

```
envFrom:
- secretRef:
    name: my-secret
```

# Command + YAML: Secret Env

```
kubectl create secret generic db-secret --from-literal=DB_USER=admin --from-literal=DB_PASS=secret123
```

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
  - name: db-app
    image: busybox
    command: ["sh", "-c", "env; sleep 3600"]
    envFrom:
    - secretRef:
        name: db-secret
```

# Lab 5: Secret as Volume

Mount secrets into containers as read-only files.

Secrets can also be mounted as volumes.

Each key in the Secret becomes a file with the secret value as content, improving access control.

# Lab 5: Secret as Volume

📁 **Secrets can be mounted into containers as read-only volumes**:

- Ensures that secret data (e.g., credentials, tokens) is not modifiable by the container at runtime.
- Ideal for applications expecting secrets in file format.

📦 **Each key in the Secret becomes a file**:

- The file name = the **key name** in the secret.
- The file content = the **value** of the key (automatically base64-decoded by Kubernetes).

📂 **Mount Path and Access**:

- Secrets are mounted at a specified path (e.g., /etc/secret).
- Best practice is to mark the volume readOnly: true.

# Lab 5: Secret as Volume

🛡 **Improved Access Control**:

File permissions can be managed at OS level.

Secret access can be restricted by pod-level RBAC and service accounts.

🧪 **Example Use Case**:

SSL certificates (e.g., tls.crt, tls.key) stored in a Secret and mounted under /etc/ssl/ path.

🧠 **Best Practices**:

Avoid logging or exposing mounted paths in app logs.

Use Kubernetes secret rotation solutions (like external secret managers) for sensitive workloads.

# Lab 5: Secret as Volume

```
volumes:
- name: secret-volume
  secret:
    secretName: my-secret
volumeMounts:
- name: secret-volume
  mountPath: /etc/secret
  readOnly: true
```

# YAML: Secret Volume

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-vol-pod
spec:
  containers:
  - name: busybox
    image: busybox
    command: ["sh", "-c", "ls /etc/secret; sleep 3600"]
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secret
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: db-secret
```